AI-Based Smart Timetable Generator Using Constraint Optimization

Complete Final-Year / Hackathon Development Blueprint

1. Project Introduction

A smart AI-powered timetable generation system for colleges that automatically creates optimized schedules while avoiding faculty clashes, room conflicts, and inefficient time allocation.

2. Problem Statement

Manual timetable generation is time-consuming, error-prone, and difficult to optimize. Colleges face issues such as faculty clashes, room conflicts, lab scheduling complexity, and inefficient distribution of subjects.

3. Objectives

• Automate timetable generation
• Avoid scheduling conflicts
• Support faculty preferences
• Optimize room/lab allocation
• Export schedules to PDF/Excel
• Provide an admin dashboard

4. Recommended Tech Stack

Frontend: HTML, CSS, JavaScript, Bootstrap
Backend: Python + Flask
Database: MySQL
AI Optimization: OR-Tools (Constraint Solver)
Version Control: Git + GitHub
Export: ReportLab, Pandas, OpenPyXL
IDE: VS Code

5. Core Features (MVP)

Admin login, Faculty management, Subject management, Room/Lab management, Class/Section management, Automatic timetable generation, Conflict detection, Weekly timetable view, Export to PDF/Excel.

6. Advanced AI Features

Faculty preferred timings, Smart load balancing, Consecutive lab scheduling, AI suggestions for conflict reduction, Multi-department optimization, Constraint-based timetable optimization.

7. Database Schema

Tables:
faculty(faculty_id, faculty_name, dept)
subjects(subject_id, subject_name, hours_per_week)
rooms(room_id, room_type, capacity)
classes(class_id, year, section)
faculty_subject(faculty_id, subject_id)
timetable(class_id, day, period, subject, faculty, room)

8. Project Architecture

Frontend → Flask Backend → Scheduling Engine → MySQL Database → Export Module

9. Folder Structure

AI_TIMETABLE_GENERATOR/
app.py
templates/
static/
database/schema.sql
scheduler/generator.py
scheduler/optimizer.py
exports/pdf_export.py

10. Timetable AI Logic

Step 1: Create empty timetable grid.
Step 2: Prioritize subjects with max hours.
Step 3: Check constraints (faculty free, room free, class free).
Step 4: Allocate slot.
Step 5: Optimize using OR-Tools.

11. Hard Constraints

No faculty clash, no room clash, no double booking, labs only in lab rooms, mandatory lunch break.

12. Soft Constraints

Faculty preference timings, avoid heavy subjects consecutively, balanced workload, preferred free periods.

13. Day-wise Roadmap

Day 1–2: Install tools, setup Flask project.
Day 3–5: Database schema design.
Day 6–8: Build login + admin dashboard.
Day 9–12: CRUD forms for faculty/subjects/rooms.
Day 13–16: Build basic timetable generator.
Day 17–20: Conflict handling.
Day 21–25: OR-Tools optimization.
Day 26–28: Export features.
Day 29–30: Testing and polishing.

14. Future Improvements

Student/faculty dashboards, mobile app, AI recommendations, cloud deployment, chatbot assistant, attendance integration, predictive scheduling.

15. Testing Strategy

Test faculty conflicts, room overlaps, empty slots, subject hour completion, lab continuity, export accuracy.

16. Resume Description

Developed an AI-powered timetable generation system using Flask, MySQL, and OR-Tools to automate academic scheduling with conflict detection and optimization.

17. Possible Viva Questions

Why Flask? Why MySQL? What is constraint optimization? Difference between hard and soft constraints? Why OR-Tools?

18. Risks & Solutions

Problem: Timetable not generated → Solution: Relax soft constraints.
Problem: Too many clashes → Solution: Add more rooms/slots.
Problem: Slow optimization → Solution: Constraint prioritization.

19. Final Checklist

Database complete ✓
CRUD complete ✓
Generator working ✓
AI optimization added ✓
Export feature added ✓

Testing completed ✓